

Naoyun SDK Python User Interface

Manual

Table of Contents

Note on Use Cases	3
Purchasing SDK	3
Frequently Asked Questions	3
1. Quick Start	4
1.1 Environment Requirements	4
1.2 Installation and Import	4
1.3 Minimal Usage Example	4
2. Core Classes and Interfaces	5
2.1 `NaoyunSdkApi` — SDK Main Entry Point	5
2.2 Algorithm Tasks	8
2.3 Callback Events	8
3. Data Models	10
3.1 `BleDeviceInfo`	10
3.2 `DeviceStateEventArgs`	10
3.3 `DeviceStatusNotificationEventArgs`	10
3.4 `DataReceivedEventArgs`	11
3.5 `SpectrumDataEventArgs`	11

3.6 `MentalStateDataEventArgs`	12
3.7 `ServerAuthResultEventArgs`	12
4. Enum Definitions	13
4.1 `ConnectionType`	13
4.2 `EarSide`	13
4.3 `SignalQuality`	13
5. Helper Functions	14
5.1 `save_eeg_to_excel`	14
6. Complete Usage Examples	14
6.1 Scan, Connect, and Receive Raw Data	14
6.2 Start the Mental State Task	15
6.3 Retrieve Filtered EEG Data and Save	16

Version: PythonSDK_1.0.1.3

Platform: Windows (x64)

Note on Use Cases

Because this product utilizes BLE data transmission, it is not suitable for ERP (Event-Related Potential) experiments that require high real-time synchronization.

Purchasing SDK

Get appID and Secret, contact: contact@brainrhythm.cn or support@veetra.ai.

Download samples: <https://github.com/brainrhythm>, currently available in Python and C# versions.

Python SDK version download link: https://github.com/brainrhythm/Naoyun_SDK_Python

C# SDK version download link: https://github.com/brainrhythm/Naoyun_SDK_Csharp

Verify the order, and we will send the appID and Secret via email.

Frequently Asked Questions

If the device cannot be found during scanning, troubleshoot as follows:

- Make sure the headset is turned on, or put it back into the charging case and take it out 3 seconds after the light turns on;
- Check whether the PC/laptop's Bluetooth supports BLE functionality;
- Check whether the headset's BLE is already connected to another device; kill the Naoyun/Veetra app on the phone from the background and then search again.

For the Python version, ensure that the library files and `naoyundemo.py` are in the same directory. Run the command in the directory where `naoyundemo.py` is located,

- for example: `python naoyundemo.py ak_XXXXXX sk_XXXXXX`

1. Quick Start

1.1 Environment Requirements

- Python 3.13 or higher (64-bit)
- OS: Windows 10/11 x64
- Dependencies (install as needed):
 - `bleak` — BLE Bluetooth functionality
 - `numpy` — Algorithms and data processing
 - `scipy` — Band-pass filtering
 - `aiohttp` — Server async authentication
 - `requests` — Server sync authentication
 - `openpyxl` — Excel file saving
 - `matplotlib` — Data visualization

1.2 Installation and Import

Place the following files from the distribution package into your project directory:

- `naoyunsdk.cp313-win_amd64.pyd` (or the `.pyd` matching your Python version)
- `libmath.dll`

Import directly in your code:

```
import naoyunsdk
from naoyunsdk import (
    NaoyunSdkApi, ServerAuthClient,
    ConnectionType, EarSide, SignalQuality,
    BleDeviceInfo, DeviceStateEventArgs,
    DeviceStatusNotificationEventArgs,
    DataReceivedEventArgs, SpectrumDataEventArgs,
    MentalStateDataEventArgs, ServerAuthResultEventArgs,
    save_eeg_to_excel
)
```

1.3 Minimal Usage Example

```
import asyncio
import datetime
from naoyunsdk import NaoyunSdkApi, EarSide
```

```

async def main():
    sdk = NaoyunSdkApi()

    # 1. Initialize (fill in your AppID and Secret)
    sdk.Initialize(app_id="YOUR_APP_ID", app_secret="YOUR_APP_SECRET", is_domestic=True)

    # 2. Scan for devices (5 seconds)
    devices = await sdk.StartBleScanAsync(datetime.timedelta(seconds=5))
    if not devices:
        print("No device found")
        return

    # 3. Connect the first device
    connected = await sdk.ConnectBleAsync(devices[0].Id)
    if not connected:
        print("Connection failed")
        return

    # 4. Start data acquisition
    await sdk.SendStartDataAsync()

    # 5. Acquire for 10 seconds
    await asyncio.sleep(10)

    # 6. Get the most recent 5 seconds of EEG data
    left_data = sdk.GetLatestEegData(EarSide.Left, seconds=5)
    right_data = sdk.GetLatestEegData(EarSide.Right, seconds=5)

    # 7. Save to Excel
    save_eeg_to_excel(left_data, right_data)

    # 8. Stop and disconnect
    await sdk.SendStopDataAsync()
    await sdk.DisconnectAsync()

asyncio.run(main())

```

2. Core Classes and Interfaces

2.1 `NaoyunSdkApi` — SDK Main Entry Point

Initialization and Version

Method	Description
<code>Initialize(app_id, app_secret, is_domestic=True)</code>	Initialize the SDK and set server authentication info. <code>is_domestic=True</code> uses the domestic server.
<code>GetSdkVersion()</code>	Returns the SDK version string.

BLE Device Scanning

Method	Description
<code>async StartBleScanAsync(timeout)</code>	Scan for BLE devices; <code>timeout</code> is a <code>datetime.timedelta</code> . Discovered devices are returned via the <code>BleDeviceDiscovered</code> callback, and the method also returns a <code>List[BleDeviceInfo]</code> when finished.
<code>async StopBleScanAsync()</code>	Stop an ongoing BLE scan.

Connection and Disconnection

Method	Description
<code>async ConnectBleAsync(bluetooth_address)</code>	Connect to the specified BLE device (pass the device address string). After a successful connection, the device status is automatically retrieved; if <code>Initialize</code> has already been called, server authentication is also performed automatically.
<code>async DisconnectAsync()</code>	Disconnect the current device.

Data Acquisition Control

Method	Description
<code>async SendStartDataAsync()</code>	Send the command to start data acquisition.
<code>async SendStopDataAsync()</code>	Send the command to stop data acquisition.
<code>async SendInitCommandAsync()</code>	Send the initialization command to retrieve the full device status.

`async GetMacAddressAsync()`

Retrieve the device MAC address. The result can be read via the `MacAddress` property.

Device Status Control

Method	Description
<code>async SetNoiseReductionModeAsync(mode)</code>	Set the noise reduction mode. <code>mode</code> : <code>0</code> =Normal, <code>1</code> =Noise Reduction, <code>2</code> =Ambient Sound. Returns <code>(send_success, verify_success)</code> .
<code>async SetTouchEnabledAsync(enabled)</code>	Set the touch switch. Returns <code>(send_success, verify_success)</code> .
<code>async SetAutoPlayEnabledAsync(enabled)</code>	Set smart playback (wear detection auto-play). Returns <code>(send_success, verify_success)</code> .

Data Reading

Method	Description
<code>GetLatestEegData(ear_side, seconds, use_filtered_data=False)</code>	Get the most recent N seconds of EEG data. <code>ear_side</code> is <code>EarSide.Left</code> or <code>EarSide.Right</code> ; <code>seconds</code> ranges from <code>0~60</code> ; <code>use_filtered_data=True</code> returns data band-pass filtered at 1–78 Hz.
<code>get_current_status()</code>	Get the current device status dictionary, including battery level, wearing status, noise reduction mode, touch switch, etc.

Properties

Property	Description
<code>IsConnected</code>	<code>bool</code> , whether a device is currently connected.
<code>MacAddress</code>	<code>str</code> or <code>None</code> , the most recently read device MAC address (uppercase hex string without separators).

2.2 Algorithm Tasks

Spectrum Analysis

Method	Description
<code>StartSpectrumTask()</code>	Start the background spectrum analysis task. Results are fed back in real time via the <code>SpectrumDataReceived</code> callback.
<code>StopSpectrumTask()</code>	Stop the spectrum task.

Real-Time Mental State

Method	Description
<code>StartMentalStateTask(interval_seconds=1.0)</code>	Start the real-time mental state feedback task. Metrics such as focus and fatigue are fed back periodically via the <code>MentalStateDataReceived</code> callback. <code>interval_seconds</code> ranges from 0.5~5.0 seconds.
<code>StopMentalStateTask()</code>	Stop the mental state task.

2.3 Callback Events

All callbacks are assignable attributes supporting both regular functions and `async` coroutine functions.

Callback Name	Parameter Type	Trigger Timing
<code>BleDeviceDiscovered</code>	<code>BleDeviceInfo</code>	When a BLE device is discovered
<code>ConnectionStateChanged</code>	<code>DeviceStateEventArgs</code>	When the connection state changes

DeviceStatusNotificationReceived	DeviceStatusNotificationEventArgs	When a device status notification is received
DataReceived	DataReceivedEventArgs	When raw EEG data is received
SpectrumDataReceived	SpectrumDataEventArgs	When spectrum analysis produces new data
SpectrumTaskStatusChanged	(bool, str)	When spectrum or mental state tasks start/stop (parameters: <i>isRunning, message</i>)
MentalStateDataReceived	MentalStateDataEventArgs	When mental state analysis produces new data
ServerAuthCompleted	ServerAuthResultEventArgs	When server authentication completes
on_log_message	str	When the SDK internally generates a log message

Callback setup example:

```
def on_device_found(device: BleDeviceInfo):
    print(f"Device found: {device.Name} [{device.Id}]")

sdk.BleDeviceDiscovered = on_device_found
```

3. Data Models

3.1 `BleDeviceInfo`

Field	Type	Description
Id	str	Device address / identifier
Name	str	Device name
BluetoothAddress	int	Bluetooth address (numeric form)
IsConnected	bool	Whether connected (default <code>False</code>)

3.2 `DeviceStateEventArgs`

Field	Type	Description
ConnectionType	ConnectionType	Connection type (currently only <code>BLE</code>)
IsConnected	bool	Whether connected
Message	str	Status description message

3.3 `DeviceStatusNotificationEventArgs`

Field	Type	Description
Command	int	Command code
EarSide	int	Ear side identifier
LeftBattery	int	Left ear battery level (0–100)
RightBattery	int	Right ear battery level (0–100)
LeftWorn	bool	Whether the left earbud is worn
RightWorn	bool	Whether the right earbud is worn
HardwareVersion	int	Hardware version number
SoftwareVersion	int	Software version number
IsBigEndian	bool	Whether big-endian

NoiseReductionMode	int	Current noise reduction mode
TouchEnabled	bool	Whether the touch switch is enabled
AutoPlayStopEnabled	bool	Whether smart playback is enabled
IsValid	bool	Whether the data is valid
ErrorMessage	str	Error message (if any)
Timestamp	datetime	Timestamp

3.4 `DataReceivedEventArgs`

Field	Type	Description
EarSide	EarSide	EarSide.Left or EarSide.Right
Data	List[float]	List of EEG data points (unit: microvolts μV)
PacketCounter	int	Packet counter
IsValid	bool	Whether the data is valid
ErrorMessage	str	Error message (if any)
Timestamp	datetime	Timestamp

3.5 `SpectrumDataEventArgs`

Spectrum data, containing energy for each frequency band for the left and right ears.

Field	Type	Description
LeftDelta / RightDelta	float	Delta wave energy
LeftTheta / RightTheta	float	Theta wave energy
LeftAlpha / RightAlpha	float	Alpha wave energy
LeftBeta / RightBeta	float	Beta wave energy
LeftGamma / RightGamma	float	Gamma wave energy
LeftLowAlpha / RightLowAlpha	float	Low Alpha energy

LeftHighAlpha / RightHighAlpha	float	High Alpha energy
LeftLowBeta / RightLowBeta	float	Low Beta energy
LeftHighBeta / RightHighBeta	float	High Beta energy
LeftLowGamma / RightLowGamma	float	Low Gamma energy
LeftHighGamma / RightHighGamma	float	High Gamma energy
LeftSignalQuality	SignalQuality	Left ear signal quality
RightSignalQuality	SignalQuality	Right ear signal quality
Timestamp	datetime	Timestamp

3.6 `MentalStateDataEventArgs`

Mental state feedback data.

Field	Type	Description
Focus	float	Focus level
Fatigue	float	Fatigue level
Relax	float	Relaxation level
Calm	float	Calmness level
Stress	float	Stress level
Timestamp	datetime	Timestamp

3.7 `ServerAuthResultEventArgs`

Field	Type	Description
IsSuccess	bool	Whether authentication succeeded
Permission	str	Permission info (permission list on success, error reason on failure)
BoundAt	str	Device binding time (if available)

LastSeen	str	Device last online time (if available)
ErrorMessage	str	Detailed error message (if any)
Timestamp	datetime	Timestamp

4. Enum Definitions

4.1 `ConnectionType`

Member	Value	Description
BLE	0	Bluetooth Low Energy

4.2 `EarSide`

Member	Value	Description
Left	0x00	Left ear
Right	0x01	Right ear

4.3 `SignalQuality`

Member	Value	Description
OK	0	Signal normal
SizeError	1	Data length abnormal
OriginalMaxValue	2	Raw data max value abnormal
OriginalZeroValue	3	Raw data zero value abnormal
FilteredMaxValue	4	Filtered max value abnormal
FilteredZeroValue	5	Filtered zero value abnormal
ThresholdError	6	Threshold abnormal

PowerFrequencyError	7	Power-line interference abnormal
OtherError	8	Other abnormality

5. Helper Functions

5.1 `save\_eeg\_to\_excel`

Save left and right ear EEG data as an Excel file (automatically falls back to CSV if `openpyxl` is not installed).

```
def save_eeg_to_excel(
    left_data: List[float],
    right_data: List[float],
    filename: Optional[str] = None,
    metadata: Optional[Dict[str, Any]] = None
) -> str
```

Parameter	Description
<code>left_data</code>	Left ear EEG data list
<code>right_data</code>	Right ear EEG data list
<code>filename</code>	Output file name; defaults to auto-generated <code>Naoyun_EEG_YYYYMMDD_HHMMSS.xlsx</code>
<code>metadata</code>	Optional metadata dictionary, written to the file header
Return value	The actual saved file path

6. Complete Usage Examples

6.1 Scan, Connect, and Receive Raw Data

```
import asyncio
import datetime
```

```

from naoyunsdk import (
    NaoyunSdkApi, EarSide, BleDeviceInfo,
    DeviceStateEventArgs, DataReceivedEventArgs
)

async def main():
    sdk = NaoyunSdkApi()
    sdk.Initialize("YOUR_APP_ID", "YOUR_APP_SECRET", is_domestic=True)

    # Set callbacks
    sdk.BleDeviceDiscovered = lambda d: print(f"Found: {d.Name}")
    sdk.ConnectionStateChanged = lambda e: print(f"Connection state: {e.IsConnected}")
    sdk.DataReceived = lambda e: print(
        f"{'Left' if e.EarSide == EarSide.Left else 'Right'} data: {len(e.Data)} points"
    )

    # Scan and connect
    devices = await sdk.StartBleScanAsync(datetime.timedelta(seconds=5))
    if devices:
        await sdk.ConnectBleAsync(devices[0].Id)
        await sdk.SendStartDataAsync()
        await asyncio.sleep(10)
        await sdk.SendStopDataAsync()
        await sdk.DisconnectAsync()

asyncio.run(main())

```

6.2 Start the Mental State Task

```

import asyncio
import datetime
from naoyunsdk import NaoyunSdkApi, MentalStateDataEventArgs

async def main():
    sdk = NaoyunSdkApi()
    sdk.Initialize("YOUR_APP_ID", "YOUR_APP_SECRET")

    sdk.MentalStateDataReceived = lambda e: print(
        f"Focus={e.Focus:.1f} Fatigue={e.Fatigue:.1f} Relax={e.Relax:.1f}"
    )

    devices = await sdk.StartBleScanAsync(datetime.timedelta(seconds=5))
    if devices and await sdk.ConnectBleAsync(devices[0].Id):
        await sdk.SendStartDataAsync()
        sdk.StartMentalStateTask(interval_seconds=1.0)
        await asyncio.sleep(30)
        sdk.StopMentalStateTask()
        await sdk.SendStopDataAsync()
        await sdk.DisconnectAsync()

```

```
asyncio.run(main())
```

6.3 Retrieve Filtered EEG Data and Save

```
import asyncio
import datetime
from naoyunsdk import NaoyunSdkApi, EarSide, save_eeg_to_excel

async def main():
    sdk = NaoyunSdkApi()
    sdk.Initialize("YOUR_APP_ID", "YOUR_APP_SECRET")

    devices = await sdk.StartBleScanAsync(datetime.timedelta(seconds=5))
    if devices and await sdk.ConnectBleAsync(devices[0].Id):
        await sdk.SendStartDataAsync()
        await asyncio.sleep(10)

        # Get raw data and 1-78 Hz filtered data
        left_raw = sdk.GetLatestEegData(EarSide.Left, 5, use_filtered_data=False)
        left_flt = sdk.GetLatestEegData(EarSide.Left, 5, use_filtered_data=True)
        right_flt = sdk.GetLatestEegData(EarSide.Right, 5, use_filtered_data=True)

        # Save
        save_eeg_to_excel(left_flt, right_flt, metadata={"subject": "demo"})

        await sdk.SendStopDataAsync()
        await sdk.DisconnectAsync()

asyncio.run(main())
```
